

# Delivery Route Optimization

E-Commerce Company – France Network

Graph Theory Project | Academic Year 2025–2026

| Member                | Signature Feature                |
|-----------------------|----------------------------------|
| Raed Meddeb           | Delivery Prediction Module       |
| Sadok Tlili           | Interactive Chatbot              |
| Ibrahim Grira         | K-Means Driver Assignment        |
| Ahmed Frouja          | Dynamic Road Weight Modifier     |
| Koussay Ibn Haj Kacem | Geographic Network Visualization |

## 1. Introduction

This project builds a delivery route optimization system for an e-commerce company operating across France. The goal is simple: help delivery drivers travel the shortest possible distances, save fuel costs, and respect delivery time windows.

We modeled 20 French cities as a graph and applied shortest-path and traversal algorithms to find optimal routes. Each team member also developed an individual signature feature that extends the core system with something extra and useful.

## 2. Data

Three CSV files make up the dataset:

| File         | What it contains     | Key columns                      |
|--------------|----------------------|----------------------------------|
| cities.csv   | 20 French cities     | city, latitude, longitude        |
| distance.csv | 31 road connections  | source, destination, distance_km |
| delivery.csv | Delivery constraints | city, priority, time window      |

The graph has 20 nodes and 30 edges after deduplication. Edge weights represent road distance in kilometres. Node coordinates are used for geographic map visualization.

## 3. Graph Model

We built an undirected weighted graph using NetworkX. Each city is a node; each road is an edge with a distance weight.

```
G = nx.Graph()
G.add_node(city, lat=latitude, lon=longitude)
G.add_edge(source, destination, weight=distance_km)
```

## 4. Algorithms

### Dijkstra – Shortest Path

Finds the minimum-distance path between two cities. At each step it visits the nearest unvisited node. Guaranteed optimal for positive edge weights.

| Route               | Optimal Path                                       | km    |
|---------------------|----------------------------------------------------|-------|
| Paris → Marseille   | Paris → Lyon → Marseille                           | 780   |
| Paris → Nice        | Paris → Lyon → Marseille → Toulon → Nice           | 995   |
| Lille → Toulouse    | Lille → Paris → Lyon → Clermont-Ferrand → Toulouse | 1 155 |
| Nantes → Strasbourg | Nantes → Paris → Reims → Strasbourg                | 875   |

### BFS – Breadth-First Search

Explores cities layer by layer from a starting point. Visits all direct neighbors before moving further out. Used here to traverse the entire network in a structured order.

From Paris: Paris → Lyon → Lille → Reims → Le Havre → Rennes → Nantes → Dijon → ...

### DFS – Depth-First Search

Follows one branch as far as possible before backtracking. Gives a different traversal order than BFS and is useful for detecting connectivity.

From Paris: Paris → Lyon → Marseille → Toulon → Nice → Grenoble → Montpellier → ...

## 5. Delivery Optimization

---

Starting from Paris, the greedy nearest-neighbor algorithm always visits the closest undelivered city next. It is not always the global optimum, but it produces fast, practical results.

| # | City       | Leg (km) | Total (km) |
|---|------------|----------|------------|
| 1 | Lyon       | 465      | 465        |
| 2 | Marseille  | 315      | 780        |
| 3 | Nice       | 215      | 995        |
| 4 | Toulouse   | 625      | 1 620      |
| 5 | Bordeaux   | 245      | 1 865      |
| 6 | Strasbourg | 1 045    | 2 910      |

Total route from Paris across 6 cities: 2 910 km.

## 6. Signature Features

### Signature 1 – Delivery Prediction Module (Raed Meddeb)

Dijkstra gives the shortest distance, but a manager also needs to know: how long will it take today, what will it cost, and should we worry about delays? This module answers all three.

The user picks a route and three conditions day of week, weather, and period. Each condition has a multiplier that reflects how traffic changes from the 90 km/h baseline:

| Condition | Value                        | Multiplier               |
|-----------|------------------------------|--------------------------|
| Day       | Friday / Sunday              | 1.45 × / 0.85 ×          |
| Weather   | Snow / Rain / Clear          | 1.65 × / 1.25 × / 1.00 × |
| Period    | Rush hour / Holiday / Normal | 1.30 × / 0.85 × / 1.00 × |

The three multipliers are combined into one traffic factor, then applied to distance:

```
traffic_factor = f(day) × f(weather) × f(period)
travel_time    = distance ÷ (90 ÷ traffic_factor)
cost           = distance × 0.15 €/km × traffic_factor
delay_risk     = LOW (<1.20) | MEDIUM (1.20-1.50) | HIGH (>1.50)
```

Example — Paris → Marseille (780 km), Monday + rush hour: factor = 1.35 × 1.00 × 1.30 = 1.76 → 15h15, 206 €, HIGH risk. Same route on Sunday: factor = 0.85 → 7h21, 99 €, LOW risk. That 107 € gap tells the manager exactly when it is worth rescheduling.

| Route             | Conditions          | Factor | Time  | Cost  | Risk |
|-------------------|---------------------|--------|-------|-------|------|
| Paris → Marseille | Mon, clear, rush hr | 1.76×  | 15h15 | 206 € | HIGH |
| Paris → Marseille | Sun, clear, normal  | 0.85×  | 7h21  | 99 €  | LOW  |
| Lille → Toulouse  | Fri, rain, rush hr  | 2.36×  | 30h17 | 409 € | HIGH |
| Nantes → Nice     | Wed, snow, normal   | 1.90×  | 25h32 | 345 € | HIGH |

### Signature 2 – Interactive Chatbot (Sadok Tlili)

A text-based chatbot that lets anyone use the system without knowing Python. The user types plain commands and gets instant answers.

| Command                | Example                                                      |
|------------------------|--------------------------------------------------------------|
| best route from A to B | best route from Paris to Nice                                |
| predict from A to B    | predict from Lille to Toulouse (then prompts for conditions) |
| bfs from <City>        | bfs from Paris                                               |
| dfs from <City>        | dfs from Lyon                                                |

The chatbot calls the same Dijkstra and prediction functions used in the core system, so results are always consistent.

### Signature 3 – K-Means Driver Assignment (Ibrahim Grira)

When a company has multiple drivers, it makes sense to divide the delivery area into geographic zones so each driver covers a compact region without crossing the whole country.

K-Means clustering groups cities by their GPS coordinates into N clusters — one per driver. The city closest to each cluster's center becomes the driver's local depot. Within their cluster, the driver's stops are ordered using the greedy nearest-neighbor method.

With 2 drivers, the algorithm splits France roughly into north and south zones, significantly reducing total travel distance compared to one driver covering everything.

### Signature 4 – Dynamic Road Weight Modifier (Ahmed Frouja)

Real roads are not static. An accident, road closure, or construction site can change the best route instantly. This feature lets the user update any edge weight on the fly — without rebuilding the graph — to simulate these situations.

```
modify_road('Paris', 'Lyon', 9999)    # simulate closure
restore_weights()                      # revert all changes
```

Demo result: when the Paris–Lyon motorway is closed (weight set to 9 999 km), Dijkstra automatically reroutes Paris → Marseille through an alternative path. Calling `restore_weights()` brings the original distances back immediately.

### Signature 5 – Geographic Network Visualization (Koussay Ibn Haj Kacem)

Instead of abstract node-and-edge diagrams, this feature uses the real GPS coordinates of each city to plot the network on a geographically accurate map. Longitude is the x-axis and latitude the y-axis, so France's shape is visible in the layout.

Two maps are produced: the full network showing all 20 cities and 30 roads, and a route-highlight map that draws any specific Dijkstra path in red on top of the grey network. This makes it easy to present results visually without any external mapping library.

## 7. Conclusion

---

The system covers all the requirements: a weighted graph model of French cities, Dijkstra shortest paths, BFS and DFS traversal, greedy delivery ordering, and five individual signature features that each add something practical on top of the core algorithms.

The signatures work together — the chatbot calls Dijkstra and the prediction module; the visualizer can highlight any route produced by the optimizer; the driver assignment uses the same greedy ordering. Everything is connected in one clean Python script.

Possible next steps: replace the greedy optimizer with a full Travelling Salesman solver, integrate a live traffic API for real-time weights, or build a web interface around the chatbot.